

Predicting Cross-Exchange Cryptocurrency Arbitrage Using Machine Learning

February 2026

Oz Cabiri Guy Harem

The Blavatnik School of Computer Science and AI

Tel-Aviv University

{ozcabiri, guyharem}@mail.tau.ac.il

Abstract

This project investigates the feasibility of predicting short-term cryptocurrency arbitrage opportunities across multiple exchanges using machine learning techniques. Market data for selected trading pairs (e.g., BTCUSD, DOGEUSD) was collected via exchange APIs and processed into minute-level features including price spreads, rolling statistics, and technical indicators such as Bollinger Bands. Multiple classification models were trained to predict whether an arbitrage opportunity would occur in the next minute. The results indicate that **LSTM architecture achieved the best performance on BTCUSD with an F1-score of 0.8242 and a ROC-AUC of 0.9691. Cross-symbol evaluation reveals that while models generalize well to high-liquidity assets like SOLUSD (CatBoost F1: 0.6845), they experience significant performance degradation on more volatile, retail-driven symbols such as DOGEUSD and LINKUSD.** Despite strong statistical signals on primary pairs, real-world deployment considerations such as latency and fees significantly impact practical profitability.

1 Introduction

1.1 Background

Cryptocurrency markets are digital platforms where assets like Bitcoin (BTC) and Ethereum (ETH) are traded against fiat currencies or other cryptocurrencies. These markets are highly fragmented, spanning dozens of exchanges worldwide, each with different liquidity, fees, and user bases. As a result, prices for the same asset can vary across platforms at any given time.

Arbitrage-buying an asset on one exchange at a lower price and simultaneously selling it on another at a higher price-exploits these temporary price discrepancies. Such opportunities arise due to market inefficiencies and the fragmented nature of cryptocurrency trading. However, predicting arbitrage

is challenging: prices are highly volatile, liquidity is uneven, and execution delays or fees can quickly erode potential profits. Machine learning offers tools to detect patterns in price movements and trading conditions, but the fleeting and stochastic nature of these opportunities makes accurate prediction a complex problem.

1.2 Problem Statement

This study investigates whether it is possible to predict short-term arbitrage opportunities across cryptocurrency exchanges. Specifically, we aim to answer the question: **Can we predict whether an arbitrage opportunity will exist in the next minute using historical cross-exchange price data?** Addressing this problem requires not only understanding price patterns but also accounting for execution constraints, trading fees, and liquidity differences that can influence the practical viability of any predicted opportunity.

1.3 Objectives

The objectives of this project are:

- 1. Collect multi-exchange price data:** Gather high-frequency trading data from multiple cryptocurrency exchanges to capture price discrepancies across assets and markets.
- 2. Engineer predictive features:** Develop features that capture market dynamics, including price spreads, volatility, and order book depth, to inform predictive models.
- 3. Train classification models:** Apply machine learning algorithms to classify whether an arbitrage opportunity is likely to occur in the next minute.
- 4. Evaluate predictive performance:** Measure model accuracy, precision, recall, and other relevant metrics to assess predictive power.

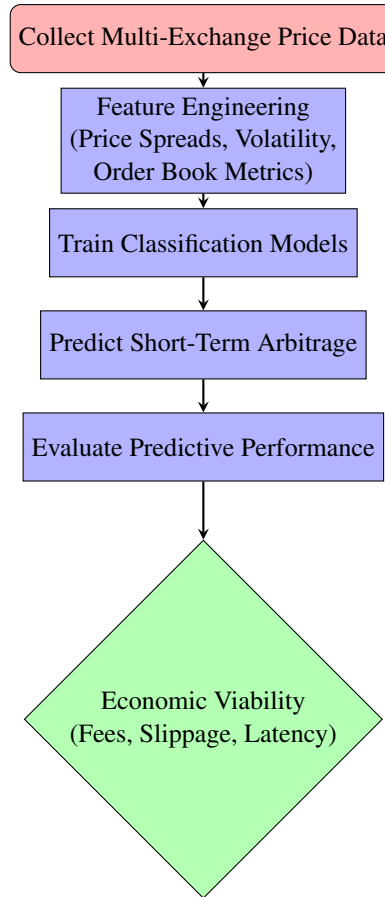


Figure 1: Pipeline for cross-exchange arbitrage prediction.

5. **Assess economic viability:** Determine whether predicted opportunities are practically profitable after accounting for trading fees, slippage, and latency.

2 Related Works

2.1 Market Efficiency and Arbitrage

The existence and persistence of arbitrage opportunities in cryptocurrency markets have been well-documented, though research indicates these gaps are narrowing. Early foundational work by [Makarov and Schoar \(2020\)](#) identified large, recurring price disparities. They attributed this market segmentation primarily to capital controls and the lack of regulatory oversight. However, more recent longitudinal studies by [Crépeillère et al. \(2023\)](#) suggest that these opportunities have decreased significantly since 2018. Their analysis indicates that increased professionalization and informed trading have driven greater market efficiency, making it increasingly difficult to generate meaningful returns from simple cross-platform strategies once transaction fees are considered.

2.2 Machine Learning for Arbitrage Prediction

We chose a similar pipeline to [Okasová et al. \(2026\)](#), utilizing OHLCV data to train models including logistic regression, random forest, SVM and neural networks. While findings highlight that MLP architectures with a 5-minute interval generally outperform simpler models in predicting profitable opportunities, we will explore the use of 1-minute intervals. Similarly, we'll use a confidence level threshold, as they were able to enhance profitability by only executing trades when the model demonstrated high certainty.

2.3 Infrastructure and Implementation

Real-time detection and execution of trades to exploit arbitrage opportunities was tried before. For example, [Oanta and Coroiu \(2023\)](#) detailed the architecture of "Crypto Advisor": a web application designed for real-time arbitrage spotting. Their work emphasizes the importance of WebSocket-driven architectures to minimize the latency between signal detection and execution. Additionally, [Gogol et al. \(2024\)](#) investigated the dynamics of

Layer-2 rollups, finding that while transactions occur more frequently than on Layer-1, arbitrage opportunities often persist for 10 to 20 blocks. These suggest that the "execution lag" will impose a challenge for real-time systems but may vary significantly depending on the underlying blockchain architecture.

2.4 Transaction Costs

A recurring theme across all relevant studies is the impact of market friction on theoretical profits. [Crépeillère et al. \(2023\)](#) demonstrated that while gross returns might appear positive, net returns frequently become negative after accounting for taker fees and withdrawal costs. [Gogol et al. \(2024\)](#) further noted that existing metrics often overestimate arbitrage potential by a large factor because they fail to account for the multi-block decay of price disparities and repeated opportunities. We will not disregard these complexities, but simplify them. The focus of this project is model performance evaluation, and the introduction of various fees and multi-block decays will introduce complexity that might obscure the results further.

3 Data Collection

3.1 Data Sources and Acquisition

The dataset for this study comprises high-frequency price data for six trading pairs, selected to provide a representative cross-section of the cryptocurrency market's liquidity and volatility profiles:

- **Market Benchmarks (BTC/USD, ETH/USD):** Included as the primary drivers of market sentiment, offering the highest liquidity and the narrowest spreads.
- **Large-Cap Alts (SOL/USD, XRP/USD, LINK/USD):** Selected for their high trading volumes and susceptibility to inter-exchange price divergence during periods of network activity.
- **Speculative Volatility (DOGE/USD):** Included to test model performance on retail-driven assets, which frequently exhibit larger, short-lived arbitrage windows.

Data was ingested from four prominent centralized exchanges (Binance, Coinbase, Bitfinex and Gate.io) covering the period from 2025-02-01 10:00 to 2025-04 -01 10:00. To navigate the

diverse API architectures, we implemented venue-specific ingestion logic. For Binance and Coinbase, we utilized high-density single-call requests (up to 10,000 candles), while Gate.io and Bitfinex required chunked pagination loops with integrated sleep intervals to comply with rate limits.

Kraken and MEXC were initially included in the acquisition phase, however they were ultimately removed. Kraken, resulted in significant data gaps that compromised temporal consistency, and MEXC only provided data for the most recent month, which had very sparse arbitrage opportunities and not enough data.

To ensure temporal consistency, an explicit chronological sorting pass was applied to Coinbase data, as its API returns results in reverse-temporal order. Finally, we implemented a standardized mapping layer to reconcile exchange-specific tickers, such as Kraken's XBT and DOG symbols, to the industry-standard BTC and DOGE.

3.2 Data Pre-processing and Standardization

Following raw acquisition, a multi-stage pipeline was implemented to transform heterogeneous exchange feeds into a synchronized, model-ready format.

3.2.1 Candle Reconstruction and Temporal Alignment

A primary challenge involved the lack of synchronization across exchange "candle closes". While the remaining venues provided native 1-minute OHLCV data, Kraken required a custom reconstruction from raw trade ticks (sub-minute timestamps). These were aggregated using pandas' groupby and agg methods with corresponding volume summation to match the granularity of other feeds, which were ultimately dropped due to significant data gaps.

All data points were converted to UTC, stripped of timezone metadata, and aligned to 1-minute boundaries using a floor("min") operation. This alignment ensures that the "Open" price for a specific minute across all six exchanges refers to the exact same 60-second window, preventing look-ahead bias and ensuring that the model only observes data that would have been simultaneously available in a live environment.

3.2.2 Schema Normalization

To achieve a uniform dataset, we applied **Reverse-Pair Normalization** as a corrective transformation. The system automatically handled reverse-pair retrieval (e.g., USD/BTC) by inverting the price ($1/\text{Price}$) and re-scaling the volume to the correct base-quote orientation.

3.2.3 Market Friction and Fee Adjustments

To bridge the gap between theoretical spreads and executable profit, every potential arbitrage opportunity was adjusted by a Friction Coefficient (γ). This coefficient incorporates an applicable Taker Fee for each venue (ranging from 0.1% to 0.6%, see Table 1 and Table 2) and a simulated latency buffer. This ensures that the spreads identified by our models are net of execution costs and slippage, addressing the realistic constraints of high-frequency trading.

Table 1: Exchange-Specific Trading Fees

Exchange	Maker Fee	Taker Fee
Binance	0.10%	0.10%
Coinbase Pro	0.40%	0.60%
Bitfinex	0.10%	0.20%
Kraken	0.16%	0.26%
Gate.io	0.15%	0.15%
MEXC	0.00%	0.20%

Table 2: Network Withdrawal/Deposit Fees

Crypto	Fee	Cost at \$54K	Percentage
BTC	0.0005 BTC	\$27	0.05%
ETH	0.005 ETH	Variable	0.08%
DOGE	Variable	Low	0.02%
USDT	\$1-25	Network Dependent	0.03%

4 Feature Engineering

The feature engineering phase aimed to translate raw OHLCV data into a multidimensional input space capable of capturing transient arbitrage opportunities. We generated 153 features, categorized into four primary domains: arbitrage-specific spreads, statistical rolling metrics, temporal features, and lagged momentum indicators.

4.1 Cross-Exchange Arbitrage Features

The core of our feature set focuses on identifying the instantaneous price divergence between venues. We calculated the *Max Close* and *Min Close* across all available exchanges to derive the *Absolute Spread* and *Spread Percentage*.

- **Execution Mapping:** Features such as *Buy Exchange* and *Sell Exchange* were encoded to identify the optimal path for capital.
- **Market Depth Proxies:** To assess the viability of a trade, we included *Volume on Buy/Sell Exchange*, *Min Volume*, and the *Volume Ratio*. These features act as filters to ensure that a theoretical spread is supported by enough liquidity to execute without excessive slippage.
- **Opportunity Modeling:** We defined *Opportunity Flags* and *Opportunity Gaps* - categorical and continuous indicators of whether the current spread exceeds the pre-defined friction threshold (γ). To eliminate complexity of taker fees and withdrawal costs, which greatly affect returns. (Cr  pelli  re et al., 2023)

4.2 Statistical and Volatility Metrics

Arbitrage opportunities often expand during periods of high market turbulence. To capture this, we engineered several statistical indicators:

- **Exchange-Specific Volatility:** Calculated for Binance, Bitfinex, Coinbase, alongside aggregate metrics like *Volatility Average*, *Max*, and *Min*.
- **Relative Price Positioning:** Features like *Price Position on Buy/Sell Exchange* and *Price Ratios* (e.g., Binance/Kraken) quantify how "extended" an exchange's price is relative to its peers.
- **Price Change Velocity:** We tracked the *Price Change* per exchange and for the specific buy/sell venues to identify which exchange is leading the price discovery process.

4.3 Technical Indicators and Trend Analysis

To understand the trend of the spread itself, we applied traditional technical analysis tools to the cross-exchange spread data over windows of 5, 15, and 30 minutes:

- **Moving Averages:** Including *Simple Moving Averages (MA)* and *Exponential Moving Averages (EMA)* for both spreads and volumes.
- **Range and Dispersion:** Features include *Rolling Standard Deviation*, *Rolling Max/Min*, and *Spread Range*.

- **Mean Reversion:** We utilized *Bollinger Bands (BB Upper/Lower)* and *BB Position* to determine if a spread is at a statistical extreme, alongside *Z-Scores* to normalize the current spread against its historical distribution.
- **Spread Dynamics:** We calculated the *Spread Rate of Change* and *Spread Acceleration* to detect the beginning of a decoupling event between exchanges.

4.4 Temporal and Lagged Features

Since arbitrage is a time-sensitive phenomenon, capturing the "memory" of the market is essential:

- **Time-Based Context:** Features like *Hour*, *Minute*, *Day of Week*, and *Is Weekend* account for variations in liquidity and volatility during different market sessions (e.g., Overlap Hours between US and Asian markets).
- **Lagged Momentum:** We generated lags at 1, 5, 10, and 30-minute intervals for key metrics, including *Spread Lag*, *Volume Lag*, and *Price Change Lag*.
- **Differential Indicators:** To capture the immediate shift in market state, we calculated the *Spread/Volume Difference from Lag*, which measures the delta between the current state and the previous 1 or 5-minute intervals.

5 Methodology

5.1 Problem Formulation and Target Definition

We treat the detection of arbitrage opportunities as a Supervised Binary Classification problem. Given a feature vector X_t representing market conditions at time t , the model objective is to predict the state of the market at $t + 1$.

The target variable y_{t+1} is defined based on a profit-maximization threshold τ :

$$y_t = \begin{cases} 1 & \text{spread}_{t+1} > \tau \\ 0 & \text{otherwise} \end{cases}$$

The threshold τ is set to 0.3 in order to exceed the aggregate cost of exchange taker fees and estimated slippage, ensuring that a positive classification corresponds to a theoretically profitable trade.

5.2 Chronological Data Partitioning

Standard randomized cross-validation is inappropriate for financial time-series data. We employed a Time-Series Split to partition our data into training, validation, and testing sets.

- **Avoidance of Data Leakage:** In a random split, the model might be trained on data from $t + 2$ to predict $t + 1$. This "look-ahead bias" provides the model with future information, leading to artificially inflated performance metrics that cannot be replicated in live trading.
- **Contextual Integrity:** By maintaining the chronological sequence, we ensure the model learns from the evolving market regimes and autocorrelation inherent in price spreads.
- **Structure:** We used the first 60% of the data for training, and the remaining 40% as an "out-of-sample" test set to evaluate real-world generalization. In RNNs (LSTM, GRU, Transformer) the 40% figure was split: 20% for hyperparameter tuning (validation), and the final [20%] as a test set.

5.3 Model Architectures

We evaluated a range of models, moving from simple linear baselines to complex non-linear learners:

5.3.1 Logistic Regression

Employed as a baseline model. It serves to establish a performance floor and evaluates the linear separability of the features.

5.3.2 Random Forest (RF)

A bagging ensemble chosen for its ability to handle high-dimensional feature spaces and capture non-linear relationships without assuming a specific data distribution.

5.3.3 XGBoost / Catboost

Selected for its performance on tabular datasets. Its boosting architecture is particularly effective at minimizing residual errors in imbalanced datasets like arbitrage feeds.

5.3.4 Neural Networks

We implemented a Multi-Layer Perceptron (MLP) architecture with ReLU activation. This allows the models (LSTM, GRU) to learn complex, high-order interactions between volume, volatility, and price across all six exchanges.

5.3.5 Transformer

To leverage the sequential nature of our 1-minute data, we implemented a Transformer architecture. Unlike traditional RNNs, the Transformer uses Self-Attention mechanisms to weigh the importance of different previous timesteps simultaneously. This allows the model to identify long-range dependencies and "regime shifts" in market volatility that might precede an arbitrage window.

5.4 Evaluation Metrics

In arbitrage modeling, the class distribution is typically highly imbalanced, as profitable opportunities are rare events. Consequently, standard Accuracy can be a misleading metric. We prioritize the following:

- **Precision:** Crucial for minimizing "False Positives." In trading, a false positive results in an executed trade that loses money due to fees.
- **Recall:** Measures the model's ability to capture as many available opportunities as possible.
- **F1-Score:** The harmonic mean of precision and recall, providing a balanced view of model reliability.
- **ROC-AUC:** Evaluates the model's ability to distinguish between classes across various probability thresholds.
- **Confusion Matrix:** Used to visualize the specific breakdown of Type I and Type II errors, which is essential for calculating the hypothetical "drawdown" of a strategy.
- **Brier Score:** Evaluates the calibration of predicted probabilities by measuring the mean squared difference between the estimated probability of an arbitrage window and the actual binary outcome. A lower score indicates that the model's confidence levels accurately reflect real-market frequencies, which is essential for determining if a predicted spread is wide enough to overcome the friction barrier of transaction fees.

5.5 Parameter Tuning

Different crypto currencies might require different model parameters in order to achieve the best prediction outcome, however, our goal is to train a single model that can generalize well enough across

various trading symbols. Thus we have decided to tune the parameters when testing on BTCUSD, and use those in other pairs as well, since BTC has the narrowest spreads, the most data, and is the primary driver of market sentiment.

6 Results

6.1 Model Performance on BTC

Table 3 summarizes the primary evaluation metrics for the best run of each architecture on the BTCUSD test set. All models significantly outperformed the Majority Class Baseline (Accuracy: 0.8572, F1: 0.0000)

6.2 Comparison of Architectures

In the sequential modeling category, the **LSTM (Run #2)** achieved the highest overall F1-score of 0.8242. The **GRU** was remarkably close, with Run #1 reaching an F1-score of 0.8224. While these recurrent models captured the highest number of profitable opportunities at a 0.5 threshold, the tree-based ensembles (**CatBoost** and **XGBoost**) exhibited superior **ROC-AUC values (>0.975)**, suggesting they are better at distinguishing between arbitrage states across the entire probability spectrum. The **Transformer** architecture proved highly unstable; while Run #5 achieved a respectable Brier score of 0.0362, most other runs failed to produce a positive F1-score.

6.3 Calibration and Threshold Sensitivity

Calibration diagnostics revealed a notable difference in how models handle "confidence." **CatBoost (Run #2)** demonstrated excellent reliability with a top-decile "gap" of only **0.050** between predicted probabilities and actual outcomes. In contrast, **XGBoost (Run #2)** was significantly overconfident, exhibiting a gap of **0.165** in its highest probability bucket. Precision-recall sweeps indicated that while the baseline F1 was strong, the economic viability of these models is highly dependent on moving the decision threshold higher (often above 0.70) to ensure that the executed spreads are wide enough to cover the friction coefficient (γ) (See Table 4).

6.4 Performance on Other Symbols

To assess the generalizability of our methodology, the best hyperparameters identified during the BTCUSD tuning phase were applied to five additional

Table 3: Performance comparison across model architectures for BTCUSD (For run parameters see Appendix 11.6)

Model Name	Best F1-score	ROC-AUC	Brier
LSTM (Run #2)	0.8242	0.9691	0.0550
GRU (Run #1)	0.8224	0.9682	0.0571
Random Forest (Run #6)	0.8175	0.9733	0.0413
CatBoost (Run #2)	0.8162	0.9758	0.0383
XGBoost (Run #6)	0.8049	0.9756	0.0425
Ridge Classifier (Run #12)	0.7848	0.9729	0.0622
Transformer (Run #5)	0.4495	0.9452	0.0362

Table 4: Model Overconfidence and Calibration (Highest Quantile Gap)

Model Name	Avg. Predicted P	Actual Positive Rate	Gap
CatBoost (Run #2)	0.695	0.645	0.050
Random Forest (Run #6)	0.689	0.646	0.043
XGBoost (Run #6)	0.766	0.644	0.122
Ridge (Run #12)	0.843	0.647	0.197

trading pairs: ETHUSD, SOLUSD, XRPUSD, DOGEUSD, and LINKUSD. The results show a clear correlation between asset liquidity and predictive performance. **SOLUSD emerged as the most predictable altcoin**, with CatBoost achieving an F1-score of **0.6845** and a strong ROC-AUC of **0.9398** (See Table 5). Similarly, high-volume assets like ETHUSD and XRPUSD maintained moderate predictability, with XGBoost reaching F1-scores of 0.4812 and 0.4683, respectively.

Table 5: Cross-Symbol Performance (CatBoost Architecture)

Symbol	F1-score	ROC-AUC	Brier
BTCUSD	0.8162	0.9758	0.0383
SOLUSD	0.6845	0.9398	0.0608
ETHUSD	0.4345	0.8880	0.0677
XRPUSD	0.3646	0.8734	0.0772
DOGEUSD	0.2033	0.8118	0.0786
LINKUSD	0.0368	0.8010	0.0322

In contrast, the models struggled significantly with more volatile or lower-liquidity assets. For DOGEUSD, the best CatBoost F1-score dropped to 0.2033, despite maintaining a respectable ROC-AUC of 0.8118, suggesting that while the model can distinguish classes, the high noise levels make precise timing difficult. The most challenging asset was LINKUSD, where CatBoost's F1-score plummeted to 0.0368. Notably, the **Transformer architecture failed to generalize across all non-BTC symbols** (See Table 6), returning an F1-score

of 0.0000 for every other pair, indicating a complete inability to find signal in the more stochastic altcoin market data. Sequential models like **LSTM and GRU demonstrated better stability** than tree ensembles on these difficult assets, with the LSTM maintaining F1-scores near 0.40 for both DOGEUSD and ETHUSD.

Table 6: Generalization Failure of Transformer Architecture

Symbol	F1-score	ROC-AUC	Brier
BTCUSD	0.4495	0.9452	0.0362
SOLUSD	0.0000	0.8826	0.0293
XRPUSD	0.0000	0.7995	0.0752
LINKUSD	0.0000	0.7533	0.1496
ETHUSD	0.0000	0.5767	0.0949
DOGEUSD	0.0000	0.5000	0.0907

7 Challenges and Technical Constraints

The implementation of a cross-exchange arbitrage model presented several significant technical and statistical hurdles. Addressing these was essential to ensure the model's results were not merely "paper profits" but representative of executable market conditions.

7.1 Data Infrastructure and Feed Integrity

The primary obstacle during the initial phase of the project was the lack of a unified infrastructure across exchange platforms. Because each venue operates as an independent entity, we were forced to

navigate a fragmented landscape of API protocols and rate-limiting policies.

For exchanges such as Gate.io we had to develop custom ingestion logic that utilized asynchronous loops. A significant technical hurdle was Kraken's trade-cursor logic, which failed to provide sufficient historical depth for the target period. This lack of data integrity necessitated its removal from the model to avoid introducing spurious signals or "broken" time-series sequences that would have skewed the results.

Beyond the logistics of acquisition, the raw data itself often lacked the integrity required for financial modeling. We encountered frequent instances of missing candles and exchange-wide downtimes, which necessitated the development of robust imputation strategies. Furthermore, inconsistent internal clock synchronization across venues required a rigorous temporal flooring process to ensure that price data was perfectly aligned at the minute-mark, preventing spurious arbitrage signals caused by time-drift.

7.2 Rare Event Detection and Class Imbalance

From a statistical perspective, predicting arbitrage is significantly more difficult than standard classification tasks because it involves identifying rare "needles in a haystack" within a system that closely resembles a random walk. In a reasonably efficient market, price divergences large enough to be profitable are anomalies, typically appearing in a very small percentage of the total observations. This extreme class imbalance poses a major risk to machine learning models, as they are naturally incentivized to minimize loss by predicting the majority class (that no opportunity exists) resulting in high accuracy but zero utility. To overcome this, we had to move away from accuracy as a primary metric and focus on threshold tuning and precision-recall optimization. This allowed the models to better learn the specific, transient market signatures that precede a decoupling event without being overwhelmed by the noise of an efficient market.

7.3 Overfitting and the Risk of Feature Leakage

The non-stationary nature of cryptocurrency markets creates a persistent risk of overfitting, where a model achieves high performance on training

data but fails to generalize to new, unseen market regimes. During development, we observed a notable "generalization gap" in complex architectures like Neural Networks and Transformers, which tended to memorize specific noise patterns or anomalies in the training set rather than learning fundamental market dynamics. Compounding this was the risk of feature leakage, a common pitfall in time-series analysis. We had to implement strict validation protocols to ensure that no information from the "future", such as the high or low price of a candle that had not yet closed, was inadvertently included in the input vector for the current prediction, as such leakage would create a deceptive but unachievable level of accuracy.

7.4 Venue-Specific Rolling Statistics and the Exchange Mixing Problem

In multi-exchange datasets, calculating rolling statistics on venue-dependent features such as "volume buy exchange" presents a unique architectural challenge. Standard rolling window functions operate sequentially across time, which inadvertently aggregates data from different exchanges when the optimal "buy" or "sell" venue shifts between observations. This phenomenon, termed the "Exchange Mixing Problem," results in a statistical "blur" where venue-specific liquidity patterns and idiosyncratic behaviors are lost in a generalized market average. This issue is further compounded by the sparse nature of individual exchange feeds, which can lead to rolling metrics being calculated from insufficient or scattered data points.

To resolve this and maintain temporal integrity, we transitioned from global rolling calculations to a venue-isolated approach. We implemented custom logic using lambda functions to identify the specific exchange involved in the current observation and then calculated rolling statistics—including moving averages and volatility metrics across 5, 15, and 30-minute windows—strictly from that specific exchange's historical data. This methodology ensures that each predictive signal is informed by the unique microstructure of the relevant exchange rather than a contaminated cross-exchange average.

While this venue-specific filtering was significantly more time-consuming during the feature engineering phase due to the computational overhead of per-row logical checks, the trade-off was essential for model performance. By providing our models with precise, uncontaminated historical context for each exchange, we were able to achieve

more accurate results.

7.5 Sequential Constraints and Chronological Partitioning

The inherent nature of sequential market data introduced two significant constraints regarding training efficiency and dataset partitioning. First, while the dataset suffered from extreme class imbalance, we were unable to apply standard random under-sampling to balance the "no-opportunity" majority class. Because our models rely on the chronological continuity of preceding minutes to learn temporal patterns and predict the state at $t+1$, dropping rows would have compromised the sequential integrity required for our lagged momentum indicators and rolling statistics. Maintaining this "chain" of data was essential to ensure the models could observe the evolving market regimes leading up to an arbitrage window.

Second, the distribution of arbitrage opportunities was non-uniform across the two-month period, with the highest density of events occurring at the beginning of the acquisition phase. In a typical machine learning workflow, data could be shuffled to ensure an even distribution of targets; however, to prevent "look-ahead bias" and ensure real-world validity, we were restricted to a strict chronological split. We chose a 60% training and 40% testing partition specifically to address this. By allocating a larger portion to the test set, we ensured that the final evaluation was conducted on a sufficiently dense collection of real opportunities, allowing us to demonstrate results on meaningful, out-of-sample data rather than over-fitting to the sparse opportunities available in a standard training window.

7.6 Transaction Costs and Economic Viability

Perhaps the most practical hurdle was the impact of transaction fees, which acted as a "friction barrier" that eliminated the majority of potential trades. In the early stages of our analysis, many identified price spreads appeared profitable on paper but resulted in a net loss once the standard fees (see Tables 1 and 2) were applied to the trade. This forced us to redefine our target variable to focus exclusively on "economic" arbitrage rather than "nominal" arbitrage. By incorporating these costs directly into our threshold selection, we ensured that the model was only incentivized to signal trades where the expected spread was wide enough to absorb both the round-trip commission and the inevitable

costs of market slippage.

In addition, in order to reduce the severity of the class imbalance we mentioned above, we decided on a γ of 0.3%, slightly below the optimistic scenario for the required fees (See Table 7).

8 Discussion

The experimental results demonstrate a **significant "Generalization Gap" between market benchmarks and altcoins**. The high performance on BTCUSD (F1: 0.8242) and SOLUSD (F1: 0.6845) suggests that in high-liquidity environments, the engineered features—specifically cross-exchange price ratios and spread momentum—capture consistent lead-lag relationships. However, the collapse of the Ridge Classifier on thinner markets, such as LINKUSD (F1: 0.1349) compared to its strong BTC performance (F1: 0.7848), indicates that the **predictive signals** in volatile assets **are highly non-linear** and cannot be captured by simple regularized models.

The failure of the Transformer (F1: 0.00 on altcoins) further confirms that **1-minute tabular market data lacks the long-range sequential dependencies** required for attention mechanisms to function effectively outside of the most stable pairs. While CatBoost remained the most robust model in terms of probability calibration, exhibiting low Brier scores across most symbols (e.g., 0.0322 for LINKUSD), its precision at the 0.5 threshold was often too low for economic viability in altcoins. This suggests that **the friction barrier is significantly higher for altcoins**, where spreads may be wider but are accompanied by higher volatility and thinner order books that increase slippage risk.

Furthermore, the calibration diagnostics across all symbols highlight a persistent trend of model overconfidence. Even in successful runs like SOLUSD, the models showed a "gap" between predicted and actual rates in the highest probability quantiles, suggesting that a **conservative threshold optimization strategy is highly important for live deployment**. Successful arbitrage prediction in these regimes depends on isolating only the most certain signals to overcome the combined impact of taker fees and market noise.

9 Limitations

9.1 Absence of Real-Time Execution

A significant theoretical challenge is the "Information Decay" that occurs between the generation of a predictive signal and the actual execution of a trade. While our model utilizes one-minute candles to predict the state of the market in the subsequent minute, real-world arbitrage often operates on a millisecond timescale. Even if a model correctly predicts a price gap, the delay involved in processing the data, generating an inference, and transmitting an order to the exchange's matching engine may result in the spread narrowing before the trade is filled. This "execution lag" means that the predictive power of a model in a backtest environment must always be viewed with caution, as high-frequency institutional participants may close the arbitrage window before a retail-level API request can be completed.

9.2 Neglect of Order Book Depth

Our analysis relies on OHLCV data, which provides a snapshot of the "top-of-book" or last-traded price, rather than the full limit order book (LOB). This assumes that the identified price is available for any volume of capital. In reality, large arbitrage trades often suffer from significant slippage as they "eat through" the available liquidity at the best bid or ask. By ignoring the depth of the order book, our results likely overestimate the capacity of these arbitrage opportunities, as executing a high-volume trade would move the price against the strategy and compress the available spread.

9.3 Withdrawal and Capital Rebalancing Delays

A critical operational hurdle in cross-exchange arbitrage is the physical movement of capital, which this study treats as instantaneous. To capture a spread between Exchange A and Exchange B, a trader must eventually rebalance their holdings to maintain the "long-short" equilibrium. In practice,

withdrawing assets from one exchange and depositing them in another involves network confirmations and exchange processing times that can range from minutes to hours. These delays trap capital and introduce "opportunity costs" that our model, which assumes perfectly positioned and liquid capital at every minute, does not account for.

9.4 Assumption of Perfect Market Liquidity

Our methodology assumes that the market remains perfectly liquid and that all orders are filled immediately at the signaled price. This "perfect liquidity" assumption fails to capture the reality of market microstructure, especially during the high-volatility events where arbitrage spreads are most common. During such periods, order books can become "thin," and exchanges may experience "lag" or "freeze" events. By assuming that every predicted opportunity is fillable, we may be ignoring the "liquidity risk" where a trade is partially filled or "orphaned" on one side, leaving the trader with an unhedged and risky directional position.

9.5 Limited Time Horizon and Regime Sensitivity

Finally, the scope of this research is limited by the specific time horizon of the dataset. Cryptocurrency markets are notoriously non-stationary and prone to rapid "regime shifts" driven by regulatory news, technological changes, or macroeconomic shifts. A model trained on a specific month of data may perform exceptionally well during that period but fail completely during a different market cycle. Because our testing period is finite, we cannot definitively claim that the identified patterns are permanent market inefficiencies rather than temporary anomalies specific to our data window.

9.6 Exclusion of Specific Venues

While this study aimed to provide a comprehensive cross-exchange view, it is limited by the exclusion of certain platforms like Kraken due to data availability constraints. The decision to prioritize

Table 7: Transaction Costs - Best Case Scenario

Cost Component	Percentage	Notes
Trading fees ($0.10\% \times 2$)	0.20%	Binance taker + Bitfinex taker
Transfer Fees	0.05%	One BTC transfer
Slippage	0.05%	Minimal market impact
Time risk	0.05%	Fast execution, low volatility
Total	0.35%	Minimum viable threshold

data density over the absolute number of exchanges means that the models may miss specific arbitrage opportunities that exist only on platforms with thinner historical records. Consequently, the findings represent a "high-integrity" subset of the market, focusing on exchanges where synchronized, continuous 1-minute data was reliably available for the entire two-month study period.

10 Conclusions

This project successfully developed and evaluated a multi-exchange pipeline for detecting short-term cryptocurrency arbitrage windows. By engineering 145 features across categories such as spreads, volatility, and lagged momentum, we established a framework that treats arbitrage detection as a supervised binary classification problem. Our methodology accounted for realistic market conditions by incorporating taker fees and simulated latency into the target definition, ensuring that models targeted economically viable opportunities rather than mere nominal price discrepancies.

The experimental findings indicate that while machine learning can reliably identify arbitrage windows in high-liquidity assets like Bitcoin (BTC) and Solana (SOL), performance is highly dependent on the specific market regime of the trading pair. Gradient-boosted trees and recurrent neural networks provided the most robust results, with the LSTM achieving the highest peak performance. However, the study also revealed the limitations of these models when applied to more volatile, retail-driven assets like Dogecoin (DOGE) and Chainlink (LINK), where predictive power significantly diminished. The total failure of complex Transformer architectures on altcoins and the degradation of linear baselines in thinner markets highlight the necessity of selecting architectures that can balance non-linear pattern recognition with the high stochasticity of cryptocurrency feeds.

Ultimately, the key insight of this research is that **successful ML-driven arbitrage requires more than statistical accuracy; it necessitates superior probability calibration and asset-specific threshold tuning**. The narrow profit margins and "friction barrier" of modern markets mean that even a well-trained model faces a high risk of executing money-losing trades if its confidence is not perfectly aligned with actual outcomes. Future work should focus on symbol-specific hyperparam-

eter optimization and the integration of order book depth data to better account for the liquidity constraints that characterize the more challenging segments of the cryptocurrency market.

11 Future Work

11.1 Transition to Reinforcement Learning

While the current study utilizes supervised learning to classify arbitrage windows, a promising avenue for future research is the implementation of *Deep Reinforcement Learning (DRL)*. Unlike traditional classifiers that provide a static "yes/no" signal, a DRL agent could learn an optimal policy for trade execution and capital management simultaneously. By defining a reward function based on the *Sharpe Ratio* or cumulative net profit, the agent could learn to navigate complex market frictions, such as when to hold a position despite a narrowing spread or how to manage execution in a way that minimizes market impact.

11.2 Real-Time Deployment and Low-Latency Infrastructure

To bridge the gap between academic backtesting and practical utility, the next phase of this project involves transitioning to a real-time deployment pipeline. This would require moving from a REST API-based polling system to a high-speed WebSocket-driven architecture capable of sub-second data ingestion. Integrating the model into a low-latency environment, possibly utilizing C++ or Rust for the execution engine, would allow for the measurement of "slippage decay," providing empirical data on how quickly an arbitrage opportunity vanishes once it is detected by a machine learning model.

11.3 Expansion to Multi-Symbol and Portfolio Arbitrage

The current methodology focuses on individual pairs in isolation. Future work could expand this to Multi-Symbol Arbitrage, where the model considers correlations and "lead-lag" effects between different assets (e.g., how a breakout in BTC/USD might precede an arbitrage window in ETH/USD). By managing a portfolio of assets simultaneously, a trading system could diversify its risk, ensuring that a lack of opportunity in one pair does not result in idle capital.

11.4 Exploration of Triangular Arbitrage

Beyond cross-exchange strategies, there is significant potential in exploring Triangular Arbitrage within a single exchange's order book. This involves exploiting price discrepancies between three different pairs. For example, *BTC/USD*, *ETH/BTC*, and *ETH/USD*. Integrating triangular arbitrage into the existing model would create a "hybrid" system that can switch between inter-exchange and intra-exchange opportunities, maximizing capital efficiency without the need for the slow and costly cross-exchange capital rebalancing discussed in our limitations.

11.5 Incorporating Order Book Depth (L2/L3 Data)

Future research should incorporate Level 2/3 market feeds to move beyond top-of-book OHLCV data. This allows for the calculation of actual liquidity and realistic slippage simulation, ensuring that identified spreads are truly executable rather than mere nominal "paper profits".

11.6 Asset-Specific Hyperparameter Optimization

The sharp performance degradation observed from BTCUSD to altcoins like LINKUSD necessitates asset-specific hyperparameter optimization. Tailoring model configurations to the unique noise and volatility profiles of retail-driven assets is essential for maintaining predictive power across diverse and thinner market regimes.

References

- Tommy Crépeillère, Matthias Pelster, and Stefan Zeisberger. 2023. [Arbitrage in the market for cryptocurrencies](#). *Journal of Financial Markets*.
- Krzysztof Gogol, Johnnatan Messias, Deborah Miori, Claudio Tessone, and Benjamin Livshits. 2024. [Layer-2 arbitrage: An empirical analysis of swap dynamics and price disparities on rollups](#). *arXiv preprint arXiv:2406.02172v1*.
- Igor Makarov and Antoinette Schoar. 2020. [Trading and arbitrage in cryptocurrency markets](#). *Journal of Financial Economics*.
- Robert-Christian Oanta and Adriana Mihaela Coroiu. 2023. [Crypto advisor: A web application for spotting cross-exchange cryptocurrency arbitrage opportunities](#). In *Proceedings of the 15th International Conference on Computer Supported Education - Volume 1: CSEDU*.

Kristína Okasová, Michal Géci, and Kristián Košťál. 2026. [Predicting arbitrage occurrences with machine learning and improved decision threshold level in live-trading crypto environments](#). *International Journal of Network Management*.

Appendix

All project files (data, code, results, models, documentation) are available at https://github.com/guyHarem/TAU_DS_Project

Table 8: xgboost parameters used

Run	Decision Threshold	N Estimators	Max Depth	Learning Rate	F1 Score
1	0.5	600	5	0.03	0.7893
2	0.5	600	5	0.02	0.7877
3	0.5	600	3	0.03	0.7714
4	0.5	300	5	0.03	0.7848
5	0.5	1200	5	0.03	0.7961
6	0.5	600	7	0.03	0.8049
7	0.5	600	5	0.05	0.7940

Table 9: catboost parameters used

Run	Decision Threshold	Iterations	Depth	Learning Rate	F1 Score
1	0.5	1000	6	0.03	0.8128
2	0.5	1000	6	0.02	0.8162
3	0.5	1000	4	0.03	0.8155
4	0.5	1000	8	0.03	0.8113
5	0.5	500	6	0.03	0.8159
6	0.5	2000	6	0.03	0.8066
7	0.5	1000	6	0.05	0.8098

Table 10: linear parameters used

Run	Model Type	Decision Threshold	Alpha	F1 Score
1	Linear	0.5	1.0	0.7341
2	Linear	0.4	1.0	0.6828
3	Linear	0.6	1.0	0.7739
4	Lasso	0.5	10.0	0.7574
5	Lasso	0.5	0.1	0.7428
6	Lasso	0.4	1.0	0.7035
7	Lasso	0.5	1.0	0.7475
8	Lasso	0.6	1.0	0.7848
9	Ridge	0.5	0.1	0.7462
10	Ridge	0.4	1.0	0.7047
11	Ridge	0.5	1.0	0.7490
12	Ridge	0.6	1.0	0.7848
13	Ridge	0.5	10.0	0.7518

Table 11: rf parameters used

Run	Decision Threshold	N Estimators	Max Depth	F1 Score
1	0.5	300	12	0.8103
2	0.5	150	12	0.8109
3	0.5	300	8	0.7975
4	0.4	300	12	0.7998
5	0.6	300	12	0.8172
6	0.5	300	16	0.8175
7	0.5	600	12	0.8096

Table 12: lstm parameters used

Run	Decision Threshold	LSTM Units	Dense Units	Dropout Rate	F1 Score
1	0.5	64	32	0.2	0.8094
2	0.5	64	32	0.3	0.8242
3	0.5	64	32	0.5	0.8129
4	0.5	32	16	0.2	0.8208
5	0.5	32	16	0.3	0.8204
6	0.5	32	16	0.5	0.8178
7	0.5	128	64	0.2	0.8041
8	0.5	128	64	0.3	0.7970
9	0.4	64	32	0.3	0.7777
10	0.6	64	32	0.3	0.8115

Table 13: gru parameters used

Run	Decision Threshold	GRU Units	Dense Units	Dropout Rate	F1 Score
1	0.5	64	32	0.2	0.8224
2	0.5	64	32	0.3	0.8132
3	0.5	64	32	0.5	0.8180
4	0.5	32	16	0.2	0.7873
5	0.5	32	16	0.3	0.8196
6	0.5	32	16	0.5	0.8180
7	0.5	128	64	0.2	0.8172
8	0.5	128	64	0.3	0.8180
9	0.4	64	32	0.2	0.7912
10	0.6	64	31	0.2	0.8100

Table 14: transformer parameters used

Run	Decision Threshold	Model Dimension	Layers	Dropout Rate	F1 Score
1	0.5	128	3	0.2	0.0000
2	0.5	128	3	0.3	0.0000
3	0.5	128	3	0.5	0.0000
4	0.5	64	2	0.2	0.0000
5	0.5	64	2	0.3	0.4495
6	0.5	64	2	0.5	0.3990
7	0.5	256	4	0.2	0.0000
8	0.5	256	4	0.3	0.0000
9	0.4	64	2	0.3	0.4290
10	0.6	64	2	0.3	0.3802

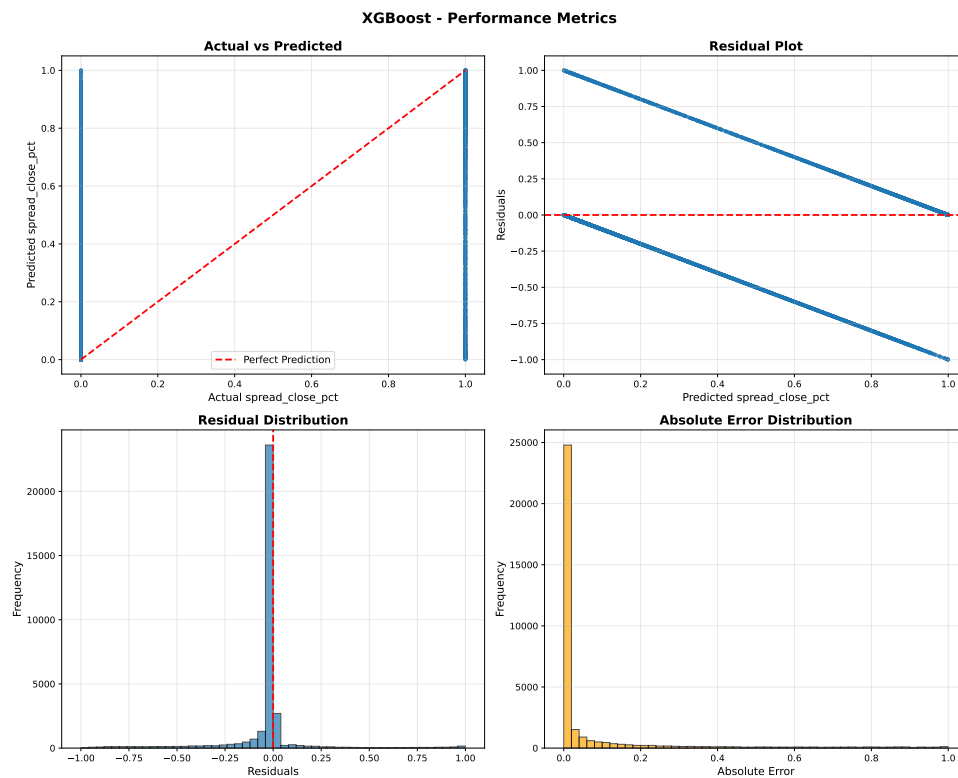


Figure 2: Results analysis for BTCUSD using xgboost

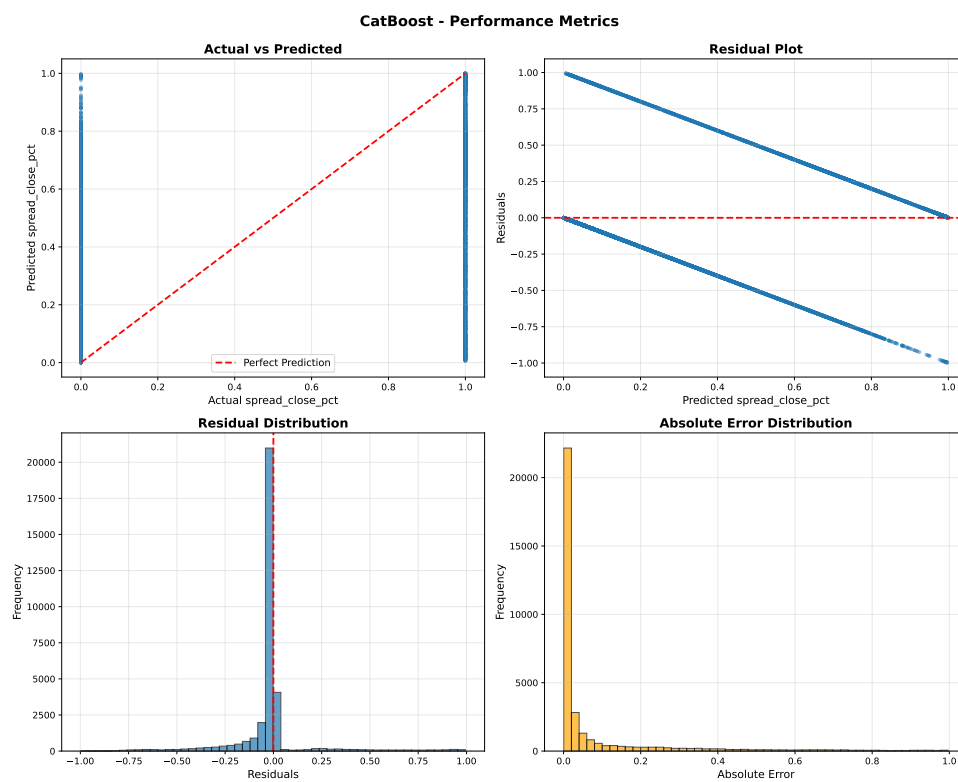


Figure 3: Results analysis for BTCUSD using catboost

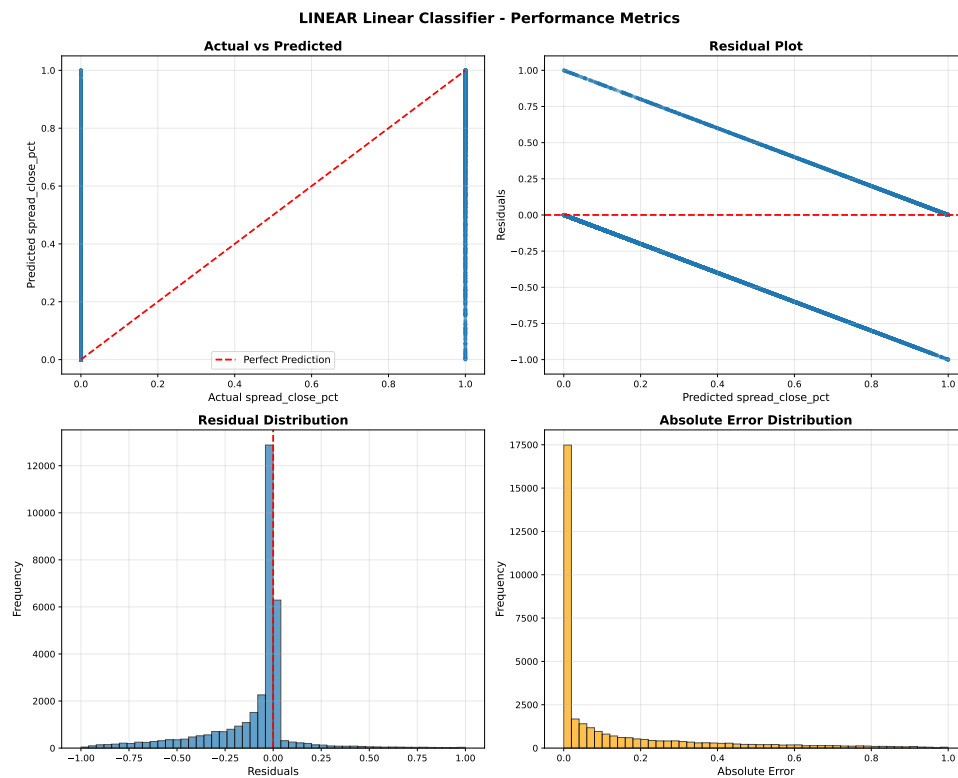


Figure 4: Results analysis for BTCUSD using linear

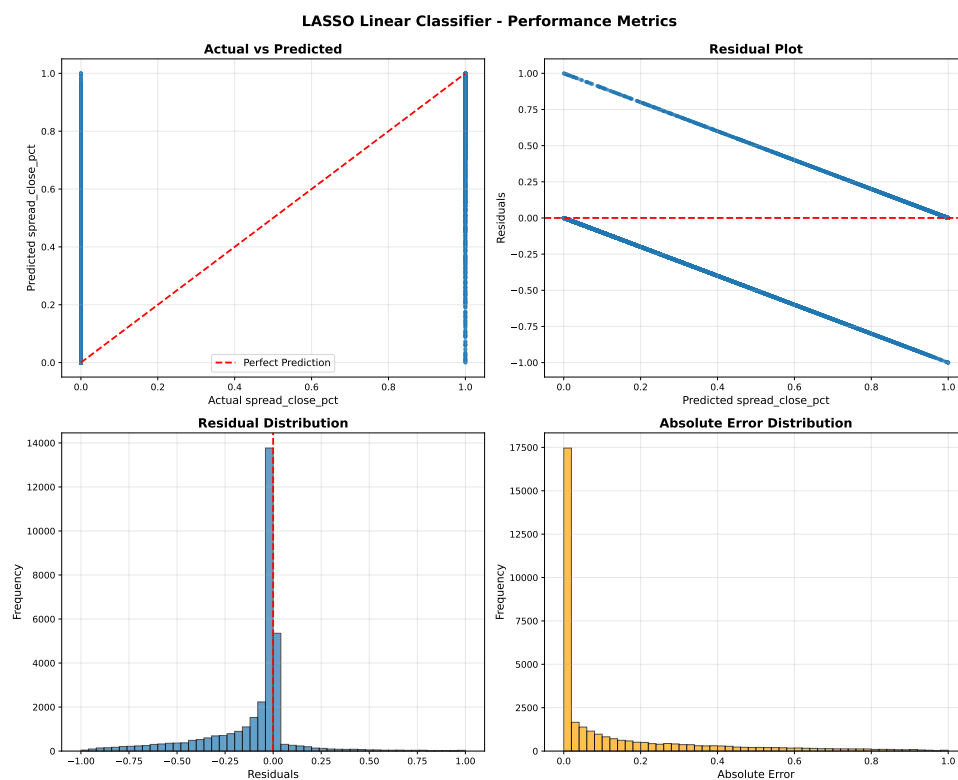


Figure 5: Results analysis for BTCUSD using lasso

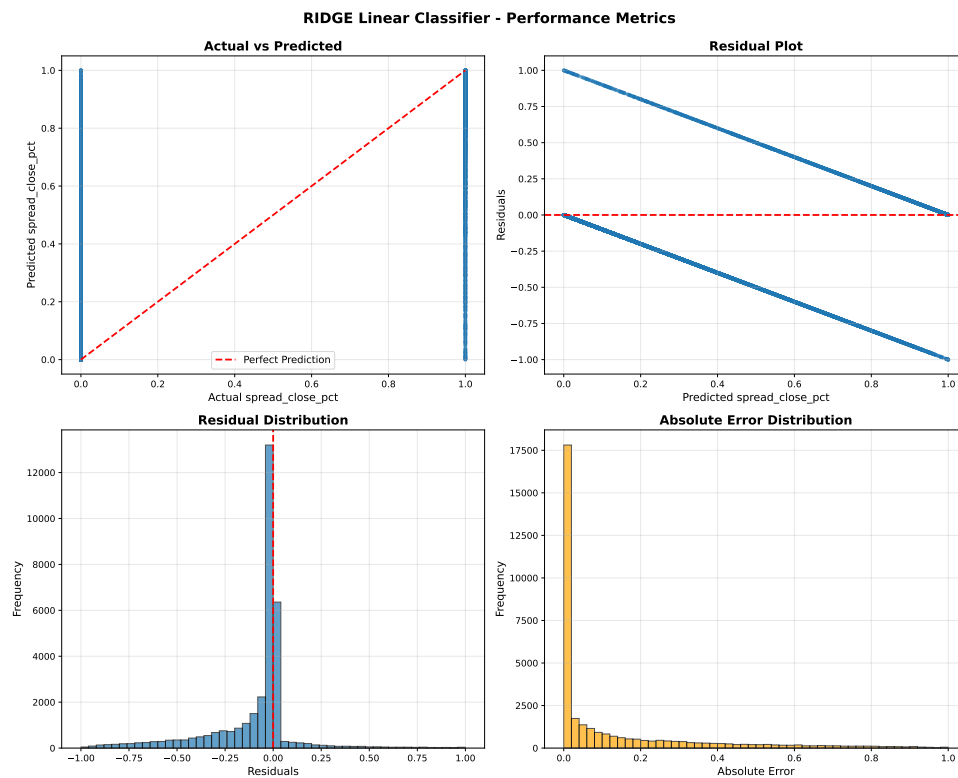


Figure 6: Results analysis for BTCUSD using ridge

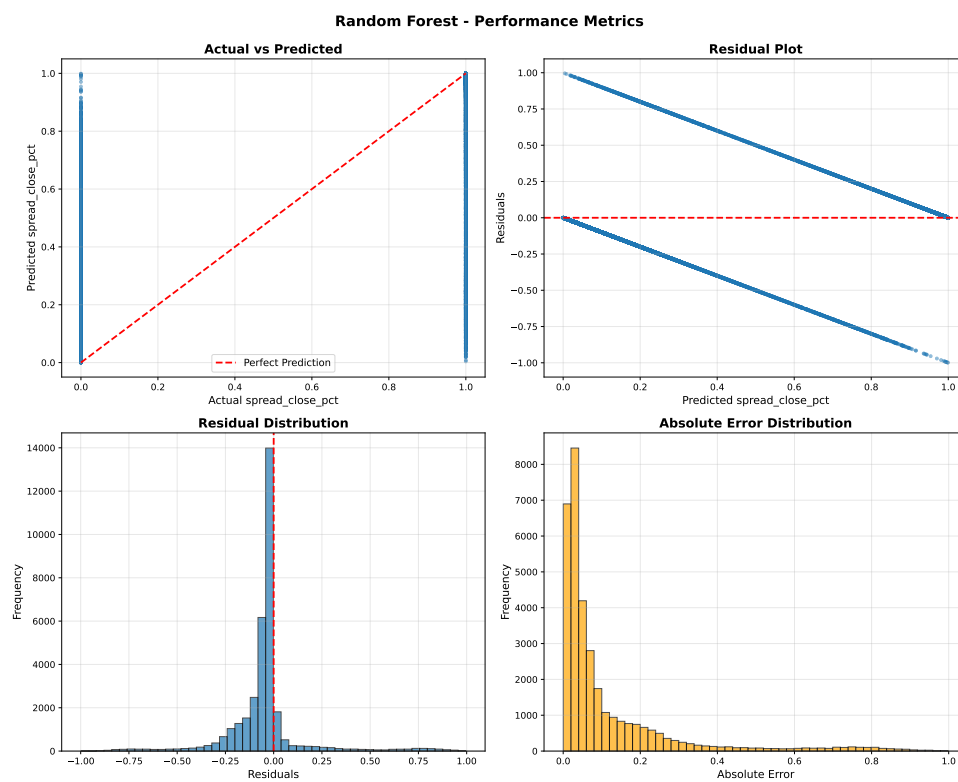


Figure 7: Results analysis for BTCUSD using rf

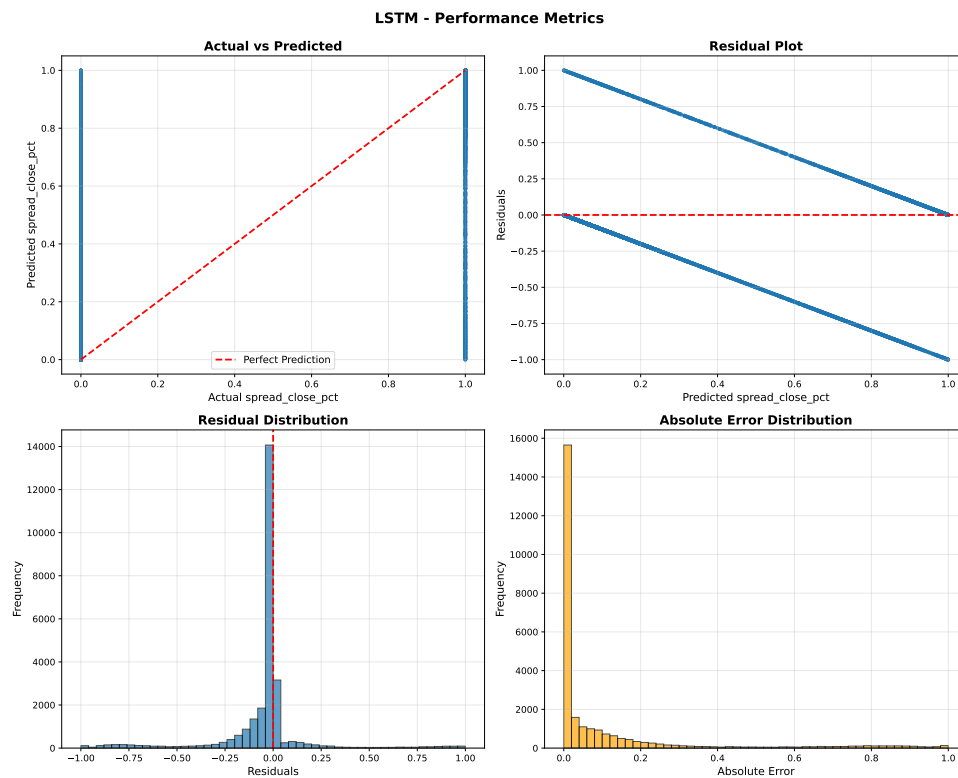


Figure 8: Results analysis for BTCUSD using lstm

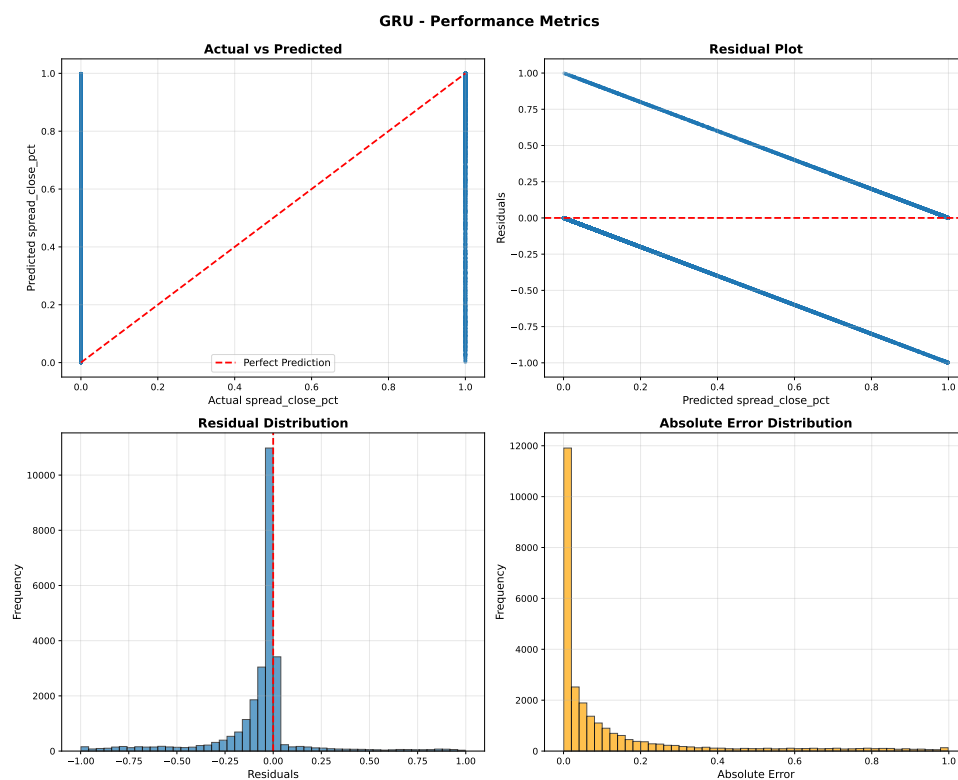


Figure 9: Results analysis for BTCUSD using gru

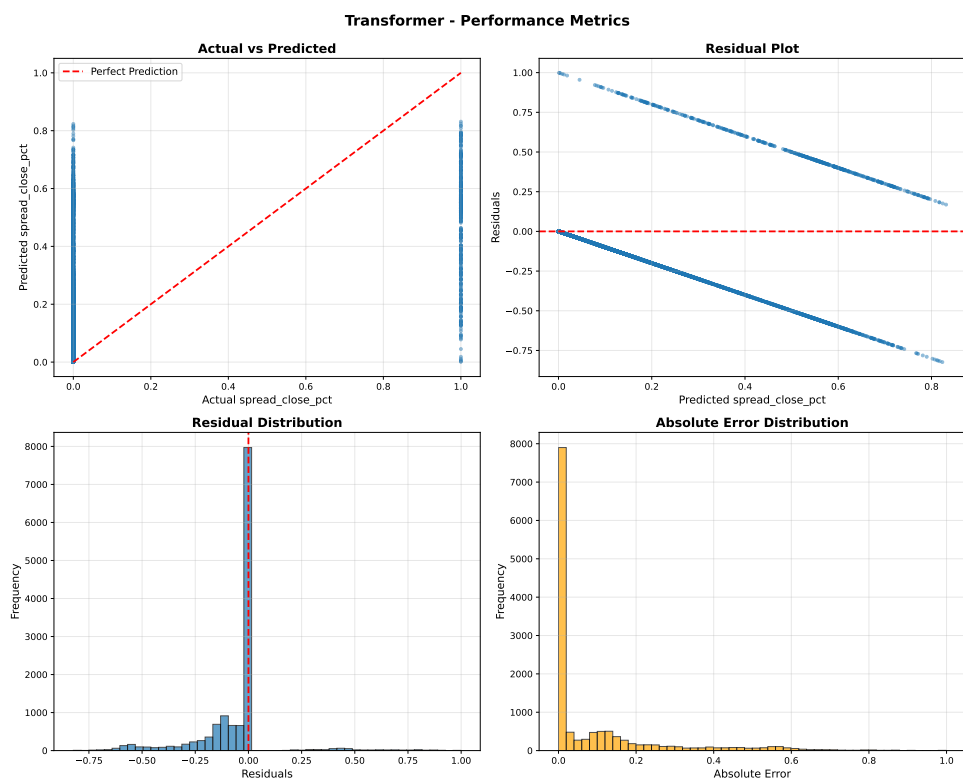


Figure 10: Results analysis for BTCUSD using transformer